

UNIVERSITÀ DEGLI STUDI DI MESSINA

DIPARTIMENTO DI SCIENZE INFORMATICHE

Progetto di di Sistemi Operativi B

*Sviluppo di un semplice orchestratore
per gestire cicli di vita di container Docker*

Studente

Antonio Venuti
Matricola 553670

Indice

1	Introduzione e Definizione del Problema	2
1.1	Il Problema	2
1.2	Sistema Distribuito	2
1.3	Macchine Virtuali	2
1.4	Docker	3
1.5	Macchine Virtuali vs Container Docker	4
2	Metodologia - Architettura di Riferimento	5
2.1	Uso dei container DinD	5
2.2	Docker Swarm	5
3	Metodologia - Progettazione dell'Orchestratore	8
3.1	Ambiente dell'orchestratore	8
3.2	Comandi e operazioni di Docker Swarm	8
3.3	Fault Tolerance in Docker Swarm	9
3.4	Garbage Collection in Docker Swarm	10
4	Implementazione	11
4.1	Struttura del sistema	11
4.2	Linguaggi e Librerie Usate	11
4.3	Deploy	11
4.4	Strutture Dati	12
4.5	Gestione dei Nodi	13
4.6	Thread: Manager	14
4.7	Thread: Failure Detection	16
4.8	Interfaccia Utente	17
5	Risultati	18
5.1	Deploy e Scaling dei Servizi	18
5.2	Stato dei nodi e dei Task	20
5.3	Creazione di Nuovi Servizi	20
5.4	Nodi in stato di Drain	21
5.5	Caduta di un nodo	23
6	Limiti e Sviluppi Futuri	24
7	Conclusioni	25
	Bibliografia	26

1 Introduzione e Definizione del Problema

1.1 Il Problema

Il progetto si propone di risolvere il seguente problema:

Progetto 7: Sviluppo di un semplice orchestratore per gestire cicli di vita di container Docker.

L'obiettivo sarà quello di creare un orchestratore, l'orchestrazione di container consiste nell'esecuzione di cluster di container su multiple macchine e ambienti. Un cluster consiste tipicamente in un gruppo di nodi (chiamati server instances). I nodi worker sono quelli che eseguono i container tramite applicazioni capaci di gestirli (come Docker).[3]

1.2 Sistema Distribuito

L'orchestratore dovrà quindi essere in grado di dare ordini ad un cluster di macchine, i cluster sono sistemi multiprocessore composti di due o più calcolatori completi – detti nodi – collegati tra loro.[1]

L'architettura da gestire sarà quindi simile ad un sistema distribuito ovvero un insieme di elaboratori fisicamente separati, e con caratteristiche spesso eterogenee, interconnessi da una rete per consentire agli utenti l'accesso alle varie risorse dei singoli sistemi.[1]

Non avendo a disposizione una infrastruttura hardware con più nodi è emersa la necessità simulare un ambiente simile ad essa. Le soluzioni principali che sono state considerate sono: Macchine Virtuali e container Docker.

1.3 Macchine Virtuali

L'idea fondamentale dietro una Macchina Virtuale è quella di astrarre l'hardware di un singolo computer (CPU, memoria, dischi, schede di rete e così via) in diversi ambienti di esecuzione differenti, creando così l'illusione che ogni ambiente distinto sia in esecuzione su un proprio computer privato.

Un'implementazione di macchina virtuale coinvolge diverse componenti. Alla base vi è l'host, il sistema hardware sottostante che gestisce le macchine virtuali. Il gestore della macchina virtuale (virtual machine manager, VMM), noto anche come hypervisor, crea e gestisce le macchine virtuali fornendo anche un'interfaccia per interagire con esse.

Di solito il processo guest è un sistema operativo. Una singola macchina fisica può quindi eseguire più sistemi operativi simultaneamente, ciascuno nella propria macchina virtuale.[1]

La virtualizzazione tramite macchine virtuali permette di gestire diversi piccoli workload su un singolo sistema host, garantendo una migliore efficienza ed abbassando i costi di gestione rispetto ad una soluzione senza virtualizzazione.

Una macchina virtuale con un suo sistema operativo e le proprie applicazioni consente ad un amministratore di sistema di deployare facilmente di provare dei proof-of-concept di applicazioni e usare ambienti di testing isolati prima di passare lo sviluppo ad un ambiente di produzione.

I vantaggi nell'uso delle macchine virtuali sono diversi:

Un primo vantaggio è la semplicità nella gestione degli asset per gestire più macchine virtuali su un numero ridotto di server, che permette di risparmiare anche sull'hardware.

Gestire le macchine virtuali vuol dire dover mantenere meno hardware, un provisioning delle risorse più veloce, un downtime ridotto e un migliore utilizzo delle risorse della macchina host.

Un altro vantaggio è il risparmio nel poter eseguire applicazioni legacy senza cambiare il sistema operativo del proprio hardware.

Infine le macchine virtuali sono altamente portabili quindi possono essere spostate da una macchina fisica ad un'altra tramite la rete.[4]

1.4 Docker

Docker è una piattaforma open source che permette a sviluppatori e amministratori di sistema di impacchettare delle applicazioni dentro dei container. Questi container possono essere spostati facilmente su una piattaforma per il deployment, come server dedicati o server cloud, per poter essere eseguiti direttamente. È possibile eseguire diversi container Docker, ognuno con la propria applicazione, su un singolo server e queste applicazioni saranno isolate l'una dall'altra provvedendo a dare più affidabilità e sicurezza dei dati.[4]

L'architettura di Docker è composta di diversi componenti:

- Docker Daemon, è il processo in background su ciascun server, desktop, workstation che ospita i container. Si occupa di interagire con i container per farli partire, fermarli, decidere l'instadamento del traffico di rete in entrata e in uscita.
- Docker Client, è lo strumento con cui gli sviluppatori e amministratori interagiscono con il Docker Daemon. Utilizzabile sia da una CLI che da interfaccia grafica.
- Container Image, un template read-only che il Docker Daemon leggere per capire come configurare e avviare il container su un server e far partire quell'applicazione sul server.
- Container Registries, sono risorse centralizzate che conservano le Docker images. Il Docker Client si occupa ordinare al Docker Daemon di accedere ad un Container Registry per ottenere l'immagine che dovrà poi rendere container e avviare.
- Container Orchestration, è il processo di gestire una grande quantità di container (centinaia, migliaia) su dozzine o centinaia di server in cloud o in un data center. Per piccoli deployment è possibile utilizzare Docker Swarm, già incluso nella piattaforma Docker.
- Container Management, mantenere i container richiede orchestrazione, scaling, load balancing, logging, sicurezza e controllo degli accessi. [4]

I container Docker sono molto simili ai container Linux, una delle caratteristiche più importanti è il loro isolamento dal resto del sistema. Per garantire questo isolamento al run di un container Docker vengono creati dei set di Namespace e Control Groups per quel container.

I Namespace sono una delle prime e più dirette forme di isolamento. Grazie al loro utilizzo i processi all'interno di un container non possono vedere o influenzare gli altri processi all'interno di un altro container o del sistema host.[5]

In Linux, un Namespace astrae una risorsa globale del sistema (tabella dei PID, file system, etc..) in modo da far apparire ai processi nello stesso namespace come se avessero la loro istanza isolata di quella risorsa globale. Cambiamenti alla risorsa globale sono visibili solo agli altri processi che sono membri di quel namespace, ma sono invisibili agli altri processi.[6]

Un'altro componente importante dei container Linux sono i Control Groups. Consentono di tracciare e limitare l'uso delle risorse del sistema usate dal container. Forniscono diverse metriche utili e aiutano a fare in modo che ogni container abbia il necessario uso di: share di memoria, CPU, I/O del disco e soprattutto fanno in modo che un singolo container non possa portare giù l'intero sistema se esaurisce una di queste risorse.[5]

In Linux, i Control Groups (cgroups) sono una funzione che permette ai processi di essere organizzati in gruppi gerarchici il cui uso delle risorse può essere limitato e monitorato. La gestione dei gruppi è implementata nel codice del kernel, mentre il monitoraggio delle risorse e la loro limitazione è implementata in dei set per tipo di risorsa in base al subsystem (ovvero uno per la memoria, uno per la CPU, e così via).[7]

1.5 Macchine Virtuali vs Container Docker

Considerando il sistema limitato su cui dovrà eseguire l'orchestratore insieme ai suoi altri nodi occorre comprendere le differenze fondamentali tra macchine virtuali e container Docker per scegliere quale dei due è più conveniente in questo caso.

Ci sono differenze importanti da considerare fra una tradizionale macchina virtuale e i container. Per quanto le macchine virtuali permettano di eseguire più sistemi operativi garantendo più sicurezza e isolamento della applicazioni, hanno comunque delle limitazioni rispetto ai container:

- Uso non efficiente delle risorse: ogni macchina virtuale richiede un intero sistema operativo per operare. Il che porta a dover usare più memoria, storage e consumo di risorse rispetto ai container.

- > I container hanno un utilizzo delle risorse più efficiente perché condividono lo stesso sistema operativo dell'host, riducendo l'uso di memoria e storage rispetto alle vm.

- Scalabilità limitata: dato che ogni macchina virtuale simula un intero computer sono richieste più risorse per gestire l'overhead e questo ne limita le capacità.

- > I container invece sono progettati per un deployment rapido di diverse istanze con minimo overhead.

- Lenti tempi di startup: accendere una macchina virtuale vuol dire caricare un intero sistema operativo, il che richiede tempo e abbassa la performance dell'intero sistema.

- > I container non avendo ciascuno un proprio sistema operativo possono partire in pochi secondi, risultando così in un più veloce deployment delle applicazioni e una migliore performance del sistema.[4]

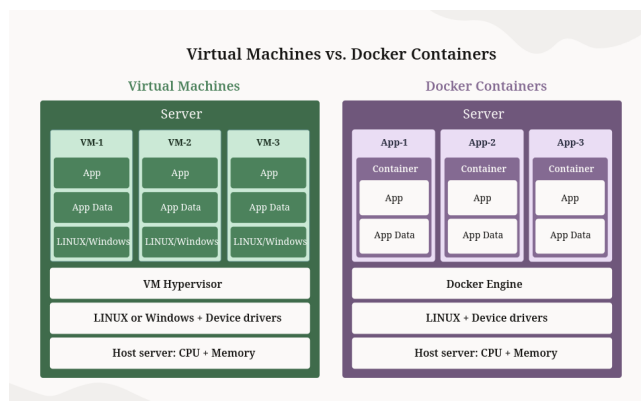


Figura 1: Differenze tra le architetture delle macchine virtuali vs container docker[4]

2 Metodologia - Architettura di Riferimento

2.1 Uso dei container DinD

L'esigenza di simulare una rete di nodi in cui ognuno è indipendente senza avere un'infrastruttura hardware adatta ha portato a valutare i vantaggi dei container rispetto alle macchine virtuali per simulare una rete di nodi. I container sono più leggeri da usare rispetto alle macchine virtuali, in quanto non richiedono di virtualizzare un intero sistema operativo. Inoltre è possibile usare i container come nodi su cui esegue Docker nel caso in cui questi siano container DinD (Docker in Docker).

Normalmente, creare un container Docker dentro Docker non sarebbe possibile perché l'applicazione ha bisogno di determinati permessi per eseguire che gli consentano di modificare i propri namespaces e cgroups. Per poterlo fare, sono stati creati i container DinD:

Docker-in-docker (DinD) è una tecnica che consente di eseguire un container Docker all'interno di un altro container Docker. DinD rende facile creare un ambiente isolato per ciascuna applicazione o servizio in fase di testing in una piattaforma CI[8] (Continuous Integration, una pratica per cui diversi sviluppatori lavorano allo stesso codice modificandolo con merge frequenti in cui la CI fa verifica e testing automatizzato).

Sebbene sembrino perfetti allo scopo, lo stesso creatore della tecnica DinD (Petazzoni) mette in guardia dall'usare i container DinD nel campo delle piattaforme CI perché richiedono di dare a questi ultimi dei privilegi da amministratore (–privileged flag) che gli consentono di modificare i namespace e cgroups allo stesso livello del Docker che viene eseguito sull'host perdendo così l'isolamento dell'host. Hanno anche altri problemi come il non poter usare le stesse immagini presenti sul docker host, che richiede il dover cancellare la cache dei container DinD e scaricare nuovamente le immagini ad ogni build che impatta sui tempi di testing della build.[9]

Petazzoni consiglia invece di usare un'altra soluzione che ora viene definita come DooD (Docker outside of Docker), questa soluzione consiste nel condividere il socket UNIX di Docker dell'host (/var/run/docker.sock) con il container tramite un montaggio di volume (-v). In questo modo, il client Docker all'interno del container non esegue un proprio demone, ma invia le istruzioni direttamente al demone dell'host. Questo farà in modo che vengano creati container siblings e non figli di quel container, ma la cache sarà condivisa tra le diverse istanze risolvendo parte del problema[9].

È stato comunque scelto di usare i container DinD, in quanto l'applicazione non verrà sviluppata su una pipeline CI in cui si susseguono molte modifiche con testing automatizzato e i container da usare saranno leggeri abbastanza da non subire problemi di performance a causa dei pull da fare durante il testing.

2.2 Docker Swarm

Dopo aver definito quali saranno i nodi che dovrà gestire l'orchestratore, analizziamo adesso Docker Swarm per avere un'architettura di riferimento su cui basare le funzionalità da implementare.

Docker include una Swarm mode per gestire nativamente un cluster di Docker Engines definito come uno "swarm". Permette di utilizzare l'interfaccia di Docker per creare uno swarm, fare deploy di servizi all'interno di uno swarm e di gestire il comportamento di quello swarm. [10]

Uno "swarm" consiste di diversi Docker host che lo eseguono in Swarm mode, ciascun Docker host può essere un manager, un worker o entrambi:

- Manager, i nodi che agiscono da manager gestiscono la membership dei nodi e la delegazione di servizi.
- Worker, si occupano di eseguire i servizi delegati dai manager.

Un "servizio" in questo caso è una definizione dei task da eseguire sui nodi. Fa parte della struttura centrale di un sistema Swarm ed è il punto di interazione dell'utente con il sistema Swarm.

Alla creazione di un servizio, è possibile specificare il numero di repliche di quel servizio, quale container image usare, che risorse assegnare, la rete, e quali comandi eseguire all'interno dei container in esecuzione.[11]

Un "task" è un container in esecuzione, che fa parte di uno swarm service ed è gestito dai swarm manager.

Se vengono create più repliche di uno stesso servizio, uno Swarm manager distribuirà il numero di task repliche specificato tra i nodi nel modo che l'utente ha specificato come desired state. Docker lavora per mantenere lo stato desiderato dall'utente, ad esempio se un worker node risulta inattivo allora Docker schedula i task di quel nodo su altri nodi worker.[11]

I passi seguiti per avviare un servizio quindi sono:

1. l'utente crea un servizio e specifica all'orchestratore quante repliche avviarne
2. l'orchestratore si occupa di avviare dei task distribuendoli tra i nodi presenti
3. se durante l'esecuzione avviene un problema con un nodo allora l'orchestratore si occuperà di ridistribuire i task che stava eseguendo su altri nodi.

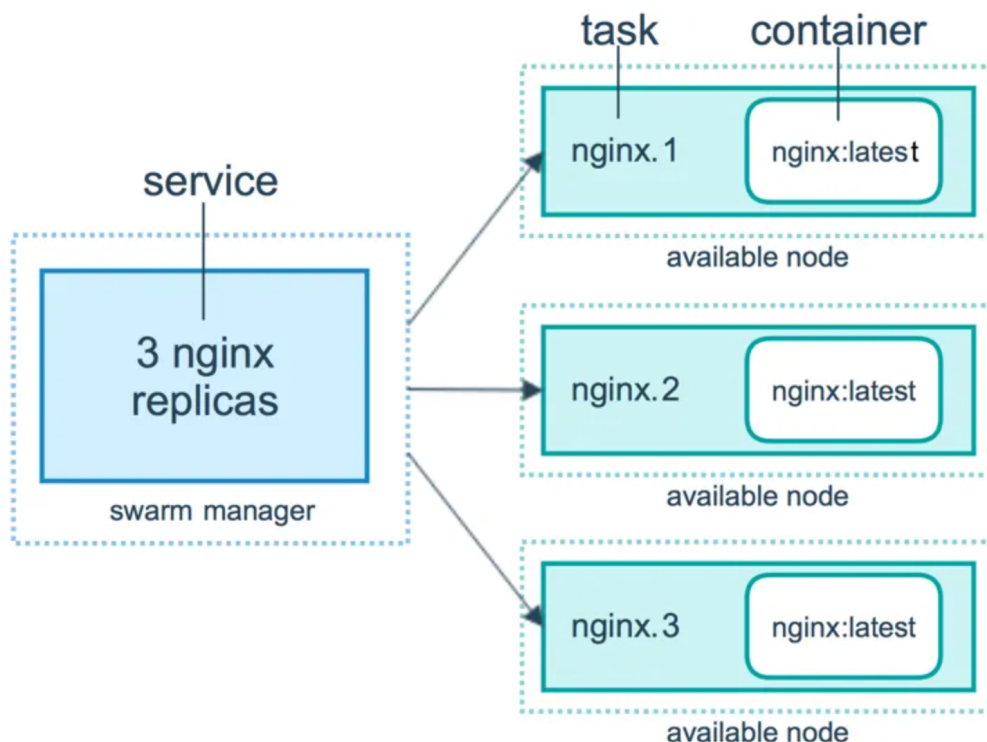


Figura 2: Servizi, task e container in Docker Swarm [12]

Le caratteristiche di Docker Swarm sono:

- Design Decentralizzato, invece di gestire la differenziazione tra nodi durante il deployment, il Docker Engine permette di specializzare a runtime. È possibile costruire un intero swarm tramite una singola disk image.
- Scaling, per ogni servizio è possibile dichiarare il numero di task da eseguire. Quando avviene lo scale-up o scale-down (ovvero si aumenta o diminuisce il numero di repliche) lo Swarm Manager si adatta aggiungendo o rimuovendo i task per mantenere lo stato desiderato.
- Riconciliazione dello stato desiderato, lo Swarm Manager monitora costantemente lo stato del cluster e riconcilia le differenze tra lo stato attuale e quello espresso nello stato desiderato dall'utente. Ad esempio, se un servizio viene impostato per essere eseguito su 10 repliche e un worker che conteneva due di queste repliche diventa inattivo allora il manager crea due nuove repliche per rimpiazzare quelle che non sono più attive. Lo swarm manager assegnerà quelle repliche a dei nodi worker che sono ancora operativi e disponibili.
- Load Balancing, è possibile esporre le porte dei servizi a un load balancer esterno. Internamente swarm permette di specificare come distribuire i servizi dei container tra i nodi.
- Sicuro di default, ogni nodo di swarm usa una autenticazione mutual TLS e crittografia per comunicazioni sicure fra se stesso e gli altri nodi.[\[10\]](#)

Nella progettazione dell'orchestratore si terrà conto di queste caratteristiche per decidere quali funzionalità aggiungere.

3 Metodologia - Progettazione dell'Orchestratore

3.1 Ambiente dell'orchestratore

Per simulare un sistema distribuito su cui l'orchestratore dovrà operare verranno usati i container DinD, ciascun container sarà un nodo worker e un ulteriore container contenente il codice dell'orchestratore sarà il nodo manager.

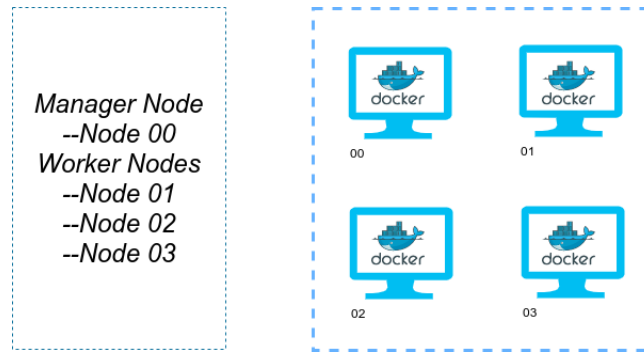


Figura 3: 1 Nodo Manager e 3 Nodi Worker [13]

3.2 Comandi e operazioni di Docker Swarm

Dato che l'architettura di riferimento è Docker Swarm, si osservano alcuni dei comandi utilizzabili tramite la CLI di Docker da poter implementare:

- `swarm init`, inizializza lo swarm. Il Container Engine che viene selezionato per questo comando diventa il manager di una nuova rete swarm di cui è il singolo nodo.
- `swarm join`, fa unire un nodo ad uno swarm. In base alla `-token` flag passata il nodo entra nel swarm come manager o come worker. [14]
- `service create`, crea un servizio con i parametri specificati. Il manager si occupa di farlo eseguire su uno dei nodi worker.
- `service ps`, stampa una lista dei task di un servizio che stanno attualmente eseguendo sul swarm.
- `service rm`, interrompe e rimuove i servizi specificati dal swarm.
- `service scale`, permette di scalare una o più repliche in un numero più basso o alto di repliche specificato. Il comando ritorna subito, ma ci vorrà del tempo per far avvenire lo scaling effettivo delle repliche. Se si vuole azzerare il numero di repliche esso si può scalare a 0. [15]
- `docker node ls`, stampa una lista dei nodi che Docker Swarm conosce.
- `docker node ps`, stampa una lista di tutti i task su un nodo che Docker Swarm conosce. [16]

- docker node update, permette di aggiornare i metadati di un nodo come la disponibilità, i suoi labels o il ruolo.

In particolare, è possibile impostare la sua disponibilità come "Drain" (–availability drain), mentre il nodo è in questo stato lo scheduler non gli assegnerà altre task. Lo scheduler spegnerà eventuali task già esistenti sul nodo e le schedulerà su altri nodi disponibili. Questa modalità è utile nel caso in cui si voglia fare manutenzione sul nodo senza necessariamente spegnerlo o scollegarlo dalla rete.[17]

3.3 Fault Tolerance in Docker Swarm

Per adempiere alle funzionalità di un orchestratore sarà necessario gestire anche i casi in cui i nodi falliscono o casi in cui i container facenti parte dei nodi si spengano occupando così spazio su disco (Garbage Collection).

Un sistema si definisce Fault Tolerant quando può tollerare un certo livello di fallimento, le fasi operative per gestire un guasto sono:

- Failure Detection, si usano dei meccanismi di heartbeat per controllare se uno dei componenti del sistema sia fallito. Un segnale di heartbeat viene mandato periodicamente da un componente ad un altro dei componenti del sistema, se entro un certo tempo non ottiene risposta allora è stato riconosciuto un fallimento.
- System Reconfiguration/Mitigation, una volta riconosciuto il fallimento bisogna riconfigurare il sistema tramite dei meccanismi di Failover. Questi meccanismi di failover possono essere di Active-Passive Failover (se un componente attivo fallisce, uno in standby prende il suo posto) o di Active-Active Failover (più componenti sono già attivi, se uno fallisce il suo carico viene distribuito agli altri).
- Recovery, una volta mitigato il problema è necessario ristabilire il componente fallito. Ciò può avvenire tramite metodi come: Rollback Recovery (dopo aver identificato un errore il sistema ritorna ad uno stato precedente tramite checkpoints o i suoi logs), Forward Recovery (il sistema prova a correggere l'errore e continuare ad operare).[2]

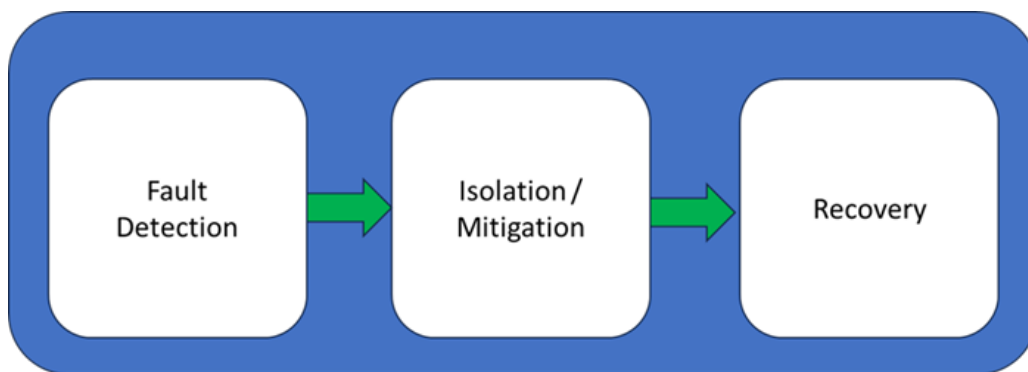


Figura 4: Ciclo di Fault Tolerance di un Sistema [18]

Docker Swarm gestisce la Fault Tolerance controllando lo stato desiderato di servizi dell'utente, nel caso in cui lo stato desiderato non corrisponda a quello attuale (ad esempio, un nodo è diventato inattivo e i suoi task non sono più in esecuzione) si occupa di distribuire il carico sugli altri nodi worker ancora in esecuzione.[19] Quindi un meccanismo di Active-Active Failover.

Nell'implementazione dell'orchestratore si terrà conto di queste funzionalità per aumentare la sua resilienza ai guasti.

3.4 Garbage Collection in Docker Swarm

In Docker Swarm un task è l'unità atomica di esecuzione che progredisce in una successione definita di stati, dalla fase di assegnazione iniziale fino al suo completamento o fallimento. Nel caso di fallimento di un task l'orchestratore crea una nuova istanza di quel task, il task precedente non viene più utilizzato e quel container rimane spento e inutilizzato su quel nodo.[\[19\]](#)

Ciò vuol dire che gli oggetti non più utilizzati come container, immagini, volumi, reti e altri occuperanno spazio sul disco senza una reale funzione. Per eliminare questi oggetti esiste il comando "prune" da eseguire localmente sul nodo, che permette di eliminare determinati oggetti non più utilizzati su quel nodo.[\[20\]](#)

I task gestiti dall'orchestratore possono essere definiti come stateless, la loro esecuzione non dipende dallo stato interno di istanze precedenti. Se si scegliesse un approccio di tipo stateful il sistema proverebbe a preservare lo stato e il file system locale del container al riavvio di quest'ultimo, introducendo il rischio di avere stati corrotti dovuti al guasto del task precedente. Questo si traduce in un comportamento simile all'uso del comando "run" per la creazione di un task rispetto ad usare il comando "start" per riavviare un container già esistente.

Nell'implementazione dell'orchestratore si terrà conto di questa caratteristica, implementando un comportamento mirato al rilevamento e alla pulizia dei task falliti sui nodi per una gestione più efficiente della memoria sui nodi worker.

4 Implementazione

4.1 Struttura del sistema

L'orchestratore è progettato per operare come nodo manager all'interno di un sistema distribuito, i nodi su cui verranno eseguiti i task saranno i nodi worker.

Questo sistema distribuito sarà simulato tramite Docker:

- nodo manager, su cui verrà eseguito l'orchestratore con cui l'utente potrà interagire sarà un container con una immagine personalizzata di Python.
- nodi worker, saranno dei container con immagini docker:dind che seguiranno le istruzioni fornite dal nodo manager.

Per focalizzare lo sviluppo sulle funzionalità di esecuzione dei task sui nodi richiesto dalle specifiche del progetto, si è scelto di avere un ambiente composto da 1 nodo manager e 3 nodi worker.

4.2 Linguaggi e Librerie Usate

Per lo sviluppo dell'orchestratore è stato scelto Python 3, che permette facilmente di comunicare con il Docker Daemon dei vari nodi utilizzando la Docker API.

Le librerie usate sono:

- docker (Docker SDK for Python): è la libreria ufficiale per interagire con il Docker Daemon in locale e in remoto. Consente di mandare istruzioni al Docker Daemon per gestire i container come il loro avvio, arresto, gestione delle reti, dei volumi, del loro stato e altro ancora.
- threading: per la gestione della concorrenza. Permette di eseguire in background il thread di monitoraggio (Failure Detection) che il thread dedicato alla gestione delle richieste dell'utente.
- queue: per implementare una coda thread-safe (FIFO) consentendo ai vari thread di accodare in modo sicuro i messaggi e le operazioni. Questo permette al thread principale di elaborare le azioni da eseguire sui nodi in modo sequenziale, evitando race condition.
- time: per la gestione dei controlli periodici sui nodi, oltre a rendere più facile per l'utente il tracciamento delle operazioni eseguite dall'orchestratore.
- uuid: per gestire identificativi univoci da associare ad ogni task creato sui nodi worker. Così da evitare conflitti di nomi tra container con nomi uguali all'interno dei nodi worker.

4.3 Deploy

Per inizializzare l'ambiente in cui l'orchestratore opera sono stati usati:

- requirements.txt, contiene le librerie esterne necessarie per eseguire il codice dell'orchestratore.
- Dockerfile, usato per la costruzione dell'immagine Docker usata dall'orchestratore. I layer specificati la rendono un'immagine Python 3.12, a cui vengono aggiunte le librerie esterne, copiato il codice dell'orchestratore e un comando per eseguirlo all'avvio del container.
- docker-compose.yaml, un file che inizializza il nodo manager con l'immagine del Dockerfile, 3 nodi worker con immagine docker:dind e connette tutti i nodi sulla stessa network.

4.4 Strutture Dati

nodo.py

```
1 class Nodo:
2     def __init__(self, nome, ip, timeout=3):
3         self.nome = nome
4         self.ip = ip
5         self.timeout = timeout
6         try:
7             self.client = docker.DockerClient(base_url=f'tcp://{ip}:2375',
8                 ↪ timeout=self.timeout)
9         except DockerException:
10            print(f"Info> Attenzione: impossibile connettersi a {nome} ({ip}) alla
11                ↪ creazione. Verrà considerato down finché non risponde.")
12            self.client = None
13            self.container_attivi = {}
14            self.disponibile = True
```

Questa classe è usata per definire una connessione con un nodo worker e gestire il suo stato durante l'esecuzione. L'attributo "ip" viene utilizzato per connettersi con il container DinD (ovvero un nodo worker), una volta stabilita la connessione viene creato un oggetto DockerClient con cui è possibile dare ordini al Docker Engine del nodo worker.

Tramite l'attributo "container_attivi" tiene traccia dei task attivi sul nodo e l'attributo "disponibile" permette agli altri componenti di sapere se il nodo può accettare nuove task o se è in stato di Drain.

I suoi metodi consentono di visualizzare le informazioni riguardo ai container attivi e spenti sul nodo.

config.py

```
1 class Config:
2     def __init__(self, image, servizio_name, command=None):
3         self.image = image
4         self.command = command
5         self.servizio_name = servizio_name
```

Questa classe è usata per tenere traccia dei servizi definiti. Per ciascun servizio assegna un nome (che sarà anche nel nome della task al suo avvio), un'immagine e opionalmente un comando da eseguire.

menu.py

```
1 class Menu:
2     def __init__(self, lista_nodi, coda, servizi_validi):
3         self.lista_nodi = lista_nodi
4         self.coda = coda
5         self.servizi = servizi_validi
```

Questa classe è usata per permettere all'utente di interagire con l'orchestratore. Contiene gli attributi necessari per mandare richieste al thread manager.

I suoi metodi sono relativi all'interazione con l'utente come creare nuovi task di un servizio, gestire lo stato dei nodi e mandare richieste al thread manager quando necessario.

4.5 Gestione dei Nodi

main.py

```
1 lista_nodi = {
2     "nodo1": Nodo("nodo1", "nodo1avo"),
3     "nodo2": Nodo("nodo2", "nodo2avo"),
4     "nodo3": Nodo("nodo3", "nodo3avo"),
5 }
6
7 servizi_validi = [
8     Config("alpine", servizio_name="alpine.sleep", command="sleep infinity"),
9     Config(image="hello-world", servizio_name="basic.hello.world"),
10 ]
```

Eseguendo l'orchestratore vengono viene inizializzato un dizionario contenente le istanze dei nodi presenti nell'infrastruttura. In questo caso l'indirizzo (hostname) dei nodi corrisponde al nome del loro container DinD, questo è possibile grazie al servizio DNS che il Docker Engine usa per la risoluzione automatica dei nomi tra container che fanno parte della stessa rete.

Viene anche creata una lista dei servizi di default che l'utente può usare all'avvio dell'orchestratore. Sono state scelte delle container image leggere per mostrare le funzionalità dell'orchestratore senza appesantire la macchina host.

Infine si occupa di avviare il thread manager e il thread failure_detection, condividendo con loro la lista dei nodi e la coda delle richieste.

cluster.py

```
1 def deploy_container(nodo, config):
2     try:
3         nodo.client.containers.run(**config)
4         print(f"\nInfo> Task: {config['image']} deployato (run) sul nodo:
5             ↳ {nodo.nome}")
6     except Exception as e:
7         print("Errore nel deploy del task: ", e)
```

Questo file contiene le funzionalità in cui viene usata la Docker API tramite i DockerClient di ciascun nodo e altre funzionalità per la gestione del cluster. Queste sono:

- `deploy_container`, fa partire una task sul nodo specificato.
- `stop_container`, ferma una task che sta venendo eseguita sul nodo specificato.
- `remove_container`, rimuove una task spenta sul nodo specificato.
- `disponibilita_nodi`, controlla la disponibilità dei nodi ritornando una lista dei nomi dei nodi attivi.
- `list_nodes`, usata per ottenere informazioni sui task attualmente attivi sui nodi e il loro stato. Ritorna un dizionario con le liste di tutti i task che stanno eseguendo su tutti i nodi della rete.

4.6 Thread: Manager

main.py

```
1 coda = queue.Queue()
2
3 thread_manager = threading.Thread(target=manager, args=(lista_nodi, coda),
4     ↪ daemon=True)
5
6 thread_manager.start()
```

All'esecuzione del codice viene inizializzato l'oggetto Queue, che permette di creare una coda FIFO thread-safe basata su meccanismi di mutua esclusione. Questa coda funge da canale asincrono in cui aggiungere richieste che il thread manager preleva ed elabora sequenzialmente.

manager.py

```
1 def manager(lista_nodi, coda):
2     while True:
3         try:
4             richiesta = coda.get(timeout=1)
5
6             try:
7                 print(f"\nManager> Schedulo ", richiesta["azione"], ' ', end="")
8                 if isinstance(richiesta["target"], Config):
9                     print(" ==> ", end='')
10                    richiesta["target"].stampa_config()
11                    attiva_nodo
12                    azione = richiesta["azione"]
13                    target = richiesta["target"]
14
15                    match azione:
16                        case "deploy":
17                            schedula_servizio(target, lista_nodi,
18                                ↪ list_nodes(lista_nodi))
19                        case "scale_down":
20                            spegni_servizio(target, lista_nodi, list_nodes(lista_nodi))
21                        case "drain_nodo":
22                            disattiva_svuota_nodo(target, lista_nodi, coda)
23                        case "attiva_nodo":
24                            attiva_nodo(target, lista_nodi)
25                        case _:
26                            print(f"Manager> Azione sconosciuta: {azione}")
27
28                    list_nodes(lista_nodi, stampa=True)
29            except Exception as e:
30                print(f"Manager> Errore nella gestione della richiesta: {e}")
31
32            finally:
33                coda.task_done()
34                rimuovi_container_spendi_tutti_nodi(lista_nodi)
35                pass
36
37            time.sleep(1)
```

Il thread manager si occupa di prelevare richieste dalla coda e di eseguirle. I tipi di richiesta che gestisce sono:

- `deploy`, quando il manager esegue un `deploy` avvia `schedula_servizio()` passando come argomenti l'oggetto `Config` del servizio da avviare, la lista dei nodi e le statistiche attuali sui task che stanno eseguendo sui nodi in questo momento. A questo punto, `schedula_servizio()` genera un nome univoco per il container e cerca nella lista di nodi quello con meno task attivi tramite la funzione `trova_nodo_least_loaded()`, infine avvia la task sul nodo scelto con `deploy_container()`.

```
1 def trova_nodo_least_loaded(lista_nodi, nodi_e_container):
2     punteggi = {}
3     for nodo, containers in nodi_e_container.items():
4         if lista_nodi[nodo].disponibile:
5             punteggio = len(containers)
6             punteggi[nodo] = punteggio
7     if not punteggi:
8         return None
9     return min(punteggi, key=lambda punteggio_nodo: punteggi[punteggio_nodo])
```

Questa funzione aggiunge una funzionalità essenziale di load balancing all'orchestratore. Prima di avviare un task, questa funzione cerca nel cluster il nodo con meno task attivi così da non sovraccaricare nodi che hanno già molti task attive su di essi e garantendo un bilanciamento dinamico durante l'esecuzione dei task.

- `scale_down`, ferma un task del servizio specificato sul nodo del cluster che ha quel tipo di task attiva e ha più task attive in quel momento. Viene riconosciuto il nodo con più task attive tramite la funzione "`trova_nodo_most_loaded`" e fermato il servizio su quel nodo.
- `drain_nodo`, disattiva il nodo selezionato impostato come in Drain e manda richieste alla coda del manager per schedulare le task sugli altri nodi ancora disponibili. Quando in stato di Drain, un nodo non viene considerato quando avviene il deploy di un task.

```
1 def disattiva_svuota_nodo(nome_nodo, lista_nodi, coda):
2     nodo = lista_nodi[nome_nodo]
3     nodo.disponibile = False
4     lista_nodi_stats = list_nodes(lista_nodi)
5     containers_da_spostare = lista_nodi_stats.get(nome_nodo, [])
6     for container in containers_da_spostare:
7         servizio_name = container["nome"].split("-")[0]
8         config_ricreata = Config(
9             image=container["immagine"],
10            servizio_name=servizio_name,
11            command=container.get("comando")
12        )
13        invia_richiesta(coda, "deploy", config_ricreata)
14        stop_container(nodo, container["id"])
15    print(f"\nManager> Nodo {nome_nodo} svuotato:
    ↳ {len(containers_da_spostare)} task rischedulati altrove.")
```

- `attiva_nodo`, riattiva un nodo in stato di Drain. Da questo momento in poi il nodo potrà tornare ad essere considerato per il deploy di task.

Nel caso in cui non ci siano richieste da gestire, si occupa della garbage collection tramite la funzione "rimuovi-container_spenti_tutti_nodi()" che controlla se ci sono task spenti sui nodi e se ci sono li rimuove in quanto non verranno usati di nuovo dall'orchestratore.

4.7 Thread: Failure Detection

main.py

```
1 thread_failure_detection = threading.Thread(target=failure_detection.heartbeat,  
↪ args=(lista_nodi, coda, 5), daemon=True)  
2 thread_failure_detection.start()
```

All'inizio dell'esecuzione, dopo il thread manager viene avviato il thread failure_detection, che si occupa di controllare periodicamente lo stato di nodi e spostare i loro task sui nodi attivi nel caso uno di loro fallisca.

failure_detection.py

```
1 def heartbeat(lista_nodi, coda, intervallo=3):  
2     while True:  
3         try:  
4             for nome, nodo in lista_nodi.items():  
5                 stato_nodo = nodo.is_up()  
6                 if stato_nodo == False:  
7                     failover(nodo.container_attivi, nodo.nome, coda)  
8         except Exception as e:  
9             print(f"Failure_detection> Errore nel thread heartbeat: {e}")  
10            time.sleep(intervallo)
```

Questo thread si occupa di controllare lo stato dei nodi periodicamente, tramite heartbeat() controlla ogni "intervallo" secondi la lista dei nodi per vedere se sono ancora attivi. Se rileva che un nodo è diventato inattivo allora questo dà inizio al meccanismo di failover. Questo aggiunge all'orchestratore una funzionalità di failure detection.

```
1 def failover(container_attivi_nodo, nome, coda):  
2     container_attivi = list(container_attivi_nodo)  
3     container_attivi_nodo.clear()  
4     if len(container_attivi) == 0:  
5         return  
6     for container in container_attivi:  
7         servizio_name = container["nome"].split("-")[0]  
8         config_ricreata = Config(  
9             image=container["immagine"],  
10            servizio_name=servizio_name,  
11            command=container["comando"])  
12        invia_richiesta(coda, "deploy", config_ricreata)  
13    print(f"\nFailure_detection> Il nodo {nome} aveva questi task attivi e verranno  
↪    rischedulati: {container_attivi}")
```

Tramite failover() si ottengono i task presenti sul nodo quando era ancora attivo, questi vengono poi schedulati tramite richieste al manager che li avvierà nel modo corretto sui nodi attivi rimasti. L'orchestratore può quindi fare una system reconfiguration che permette al sistema di continuare ad eseguire task anche nel caso di un eventuale guasto di un nodo.

4.8 Interfaccia Utente

main.py

```
1 menu = Menu(lista_nodi, coda, servizi_validi)
2 menu.avvia_menu()
```

All'avvio del programma, dopo aver avviato i due thread precedenti, viene istanziato un oggetto Menu e poi avviato tramite un suo metodo per mostrare l'interfaccia all'utente.

menu.py

Il menù mostrato all'utente si compone di tre schermate:

- `avvia_menu()`, da cui è possibile accedere alle altre due schermate, aggiungere alla lista `servizi_disponibili` la configurazione di un servizio personalizzabile dall'utente e uscire dal programma.
- `menu_servizi()`, da cui è possibile scegliere di creare una replica di un servizio a scelta dell'utente fra la lista `servizi_disponibili` oppure scalare (aumentare, diminuire, azzerare) il numero di repliche di un servizio scelto dall'utente.
- `menu_stato_nodi()`, da cui è possibile monitorare lo stato attuale dei nodi e i task eseguiti attualmente su ciascun nodo. Inoltre è possibile impostare un nodo in stato Drain o riattivarlo.

Le scelte dell'utente che riguardano i task o il cambio di stato di un nodo causano l'aggiunta di una o più richieste nella coda del thread manager, così che vengano eseguite sequenzialmente quando arriverà il loro turno nella coda.

A supporto dell'interfaccia, la classe usa altri metodi che si occupano: del conteggio delle repliche attive e tradurre le opzioni selezionate in una sequenza di richieste che verranno inoltrate alla coda per l'elaborazione da parte del thread manager.

utils.py

```
1 def scegli_config():
2     immagine = input("Che immagine vuoi usare? Scrivi il nome di un immagine
   ↳ esistente di cui ogni nodo potrà fare il pull\n")
3     servizio_name = input("Che nome vuoi dare a questo servizio custom? ...")
4     if "-" in servizio_name:
5         return None
6     comando = input("Che comando vuoi usare sul container? Lascia vuoto per nessun
   ↳ comando\n")
7     return Config(immagine, servizio_name, command=comando)
```

Il modulo `utils.py` raccoglie funzioni che forniscono operazioni di supporto al resto del sistema. Tra queste ci sono:

- `scegli_config()`, che permette all'utente di dare un nome ad un servizio e scegliere per quest'ultimo un'immagine e un comando da eseguire.
- `is_container_vecchio()`, che serve per verificare se un task all'interno di un nodo è in stato di "created" da troppo tempo e quindi non si avvierà. Questo può succedere nel momento in cui un nodo non riesce ad avviare una task perché un comando non è adatto a quella container image o altri motivi.

5 Risultati

5.1 Deploy e Scaling dei Servizi

La prima voce del menù permette di avviare una singola istanza di un servizio, questa verrà schedulata sul nodo meno carico del cluster:

```
1  ---Menu Servizi---
2  Scegli una voce dal menù:
3  1) Crea replica di 1 servizio
4  2) Scala N repliche di un servizio
5  3) Torna al Menu Principale
6  1
7  Scegli quale servizio avviare tra quelli disponibili:
8  1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
9  2) Immagine: hello-world | Nome Servizio: basic.hello.world
10 1
11 Manager> Schedulo  deploy  ==> Immagine: alpine | Nome Servizio: alpine.sleep |
    ↳ Comando: sleep infinity
12 Info> Task: alpine deployato (run) sul nodo: nodo1
13 nodo1 | UP | Task attivi(1):
14 nome : alpine.sleep-9c050d42b7 | id : 0a0d9d183285 | immagine : alpine:latest |
    ↳ comando : sleep infinity |
15 nodo2 | UP | Task attivi(0):
16 nodo3 | UP | Task attivi(0):
```

La seconda voce del menù permette di avviare una operazione di scaling, così che l'utente possa impostare il numero desiderato di repliche per un servizio e il sistema possa ricalibrare il numero di task in base alla scelta dell'utente:

```
1  ---Menu Servizi---
2  Scegli una voce dal menù:
3  1) Crea replica di 1 servizio
4  2) Scala N repliche di un servizio
5  3) Torna al Menu Principale
6  2
7  Scegli quale servizio vuoi scalare
8  1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
9  2) Immagine: hello-world | Nome Servizio: basic.hello.world
10 1
11 Scegli il numero di repliche che vuoi avere per il servizio scelto
12 4
13 nodo1 | UP | Task attivi(2):
14 nome : alpine.sleep-c468f0e906 | id : 6197bda48875 | immagine : alpine:latest |
    ↳ comando : sleep infinity |
15 nome : alpine.sleep-9c050d42b7 | id : 0a0d9d183285 | immagine : alpine:latest |
    ↳ comando : sleep infinity |
16 nodo2 | UP | Task attivi(1):
17 nome : alpine.sleep-0e9073b0fe | id : ae962553a84b | immagine : alpine:latest |
    ↳ comando : sleep infinity |
18 nodo3 | UP | Task attivi(1):
19 nome : alpine.sleep-ea97c1f21d | id : fbd19afd6887 | immagine : alpine:latest |
    ↳ comando : sleep infinity |
```

La seconda voce del menù permette anche lo scale down di un servizio, durante questo processo vengono scelti i nodi che hanno quel servizio con più task attivi in quel momento e se possibile si tolgono più task da quel nodo:

```
1  ---Menu Servizi---
2  Scegli una voce dal menù:
3  1) Crea replica di 1 servizio
4  2) Scala N repliche di un servizio
5  3) Torna al Menu Principale
6  2
7  Scegli quale servizio vuoi scalare
8  1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
9  2) Immagine: hello-world | Nome Servizio: basic.hello.world
10 1
11 Scegli il numero di repliche che vuoi avere per il servizio scelto
12 2
13 (...)
14 nodo1 | UP | Task attivi(0):
15 nodo2 | UP | Task attivi(1):
16 nome : alpine.sleep-0e9073b0fe | id : ae962553a84b | immagine : alpine:latest |
   ↳ comando : sleep infinity |
17 nodo3 | UP | Task attivi(1):
18 nome : alpine.sleep-ea97c1f21d | id : fbd19afd6887 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
```

Infine, la seconda voce permette anche di azzerare le repliche di un servizio:

```
1  ---Menu Servizi---
2  Scegli una voce dal menù:
3  1) Crea replica di 1 servizio
4  2) Scala N repliche di un servizio
5  3) Torna al Menu Principale
6  Manager> Task già spenti rimossi su nodo1: ['alpine.sleep-c468f0e906,
   ↳ alpine:latest', 'alpine.sleep-9c050d42b7, alpine:latest']
7  2
8  Scegli quale servizio vuoi scalare
9  1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
10 2) Immagine: hello-world | Nome Servizio: basic.hello.world
11 1
12 Scegli il numero di repliche che vuoi avere per il servizio scelto
13 0
14 Richieste 2 rimozioni di repliche di 'alpine.sleep'
15 Manager> Schedulo scale_down ==> Immagine: alpine | Nome Servizio: alpine.sleep
   ↳ | Comando: sleep infinity
16 Info> Un task sul nodo nodo2 è stato fermato
17 Manager> Schedulo scale_down ==> Immagine: alpine | Nome Servizio: alpine.sleep
   ↳ | Comando: sleep infinity
18 Info> Un task sul nodo nodo3 è stato fermato
19 nodo1 | UP | Task attivi(0):
20 nodo2 | UP | Task attivi(0):
21 nodo3 | UP | Task attivi(0):
```

5.2 Stato dei nodi e dei Task

Tramite le prime due voci del Menù Stato Nodi è possibile visualizzare lo stato dei nodi e i task in esecuzione (o terminati) su ciascun nodo del cluster:

```
1  ---Menu Stato Nodi---
2  Scegli una voce dal menù:
3  1) Stampa stato e task attivi sui nodi
4  2) Stampa stato e task attivi e spenti sui nodi
5  3) Svuota un nodo
6  4) Attiva un nodo se è in DRAIN
7  5) Torna al Menu Principale
8  1
9  nodo1 | UP | Task attivi(1):
10 nome : alpine.sleep-7a2ee9f608 | id : 5d5909aed749 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
11 nodo2 | UP | Task attivi(0):
12 nodo3 | UP | Task attivi(0):
```

5.3 Creazione di Nuovi Servizi

Con la terza voce del menù principale è possibile creare una nuova configurazione per un servizio, definendo il nome, l'immagine e il comando di esecuzione:

```
1  ---Menu Principale---
2  Scegli il numero di una voce dal menu:
3  1) Menu - Servizi
4  2) Menu - Stato Nodi
5  3) Aggiungi Configurazione Servizio
6  4) Uscita
7  5) Uscita Forzata (senza finire di gestire le richieste in coda)
8  3
9  Che immagine vuoi usare? Scrivi il nome di un immagine esistente di cui ogni nodo
   ↳ potrà fare il pull
10 busybox
11 Che nome vuoi dare a questo servizio custom? (Non usare nomi con il trattino '-')
12 busyboxsleep
13 Che comando vuoi usare sul container? Lascia vuoto per nessun comando
14 sleep infinity
15 ---Menu Servizi---
16 Scegli una voce dal menù:
17 1) Crea replica di 1 servizio
18 (...)
19 1
20 Scegli quale servizio avviare tra quelli disponibili:
21 1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
22 2) Immagine: hello-world | Nome Servizio: basic.hello.world
23 3) Immagine: busybox | Nome Servizio: busyboxsleep | Comando: sleep infinity
24 3
25 (...)
26 nodo2 | UP | Task attivi(1):
27 nome : busyboxsleep-5ee4effa6f | id : 07ac0ad6fde8 | immagine : busybox:latest |
   ↳ comando : sleep infinity |
```

5.4 Nodi in stato di Drain

Tramite la terza voce del Menù Stato Nodi è possibile svuotare un nodo mettendolo in Drain e facendo passare i suoi task in esecuzione su altri nodi:

```
1  ---Menu Stato Nodi---
2  Scegli una voce dal menù:
3  1) Stampa stato e task attivi sui nodi
4  2) Stampa stato e task attivi e spenti sui nodi
5  3) Svuota un nodo
6  4) Attiva un nodo se è in DRAIN
7  5) Torna al Menu Principale
8  3
9  Scegli quale nodo vuoi svuotare (andrà in stato di DRAIN)
10 1) nodo1
11 2) nodo2
12 3) nodo3
13 3
14 Manager> Schedulo  drain_nodo
15 Manager> Nodo nodo3 svuotato: 1 task rischedulati altrove.
16 nodo1 | UP | Task attivi(2):
17 nome : alpine.sleep-3ea3bf8e09 | id : 1d9a09c5a8e9 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
18 nome : alpine.sleep-7a2ee9f608 | id : 5d5909aed749 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
19 nodo2 | UP | Task attivi(2):
20 nome : alpine.sleep-17e1ac63be | id : a06a19e76217 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
21 nome : busyboxsleep-5ee4effa6f | id : 07ac0ad6fde8 | immagine : busybox:latest |
   ↳ comando : sleep infinity |
22 nodo3 | UP | Task attivi(0):
23 (...)
24 Nodi in stato di DRAIN: nodo3
```

Mentre un nodo è in Drain non verrà considerato nell'assegnazione di nuovi task, restando vuoto fino a che non viene riattivato:

```
1  ---Menu Servizi---
2  Scegli una voce dal menù:
3  1) Crea replica di 1 servizio
4  2) Scala N repliche di un servizio
5  3) Torna al Menu Principale
6  1
7  Scegli quale servizio avviare tra quelli disponibili:
8  1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
9  2) Immagine: hello-world | Nome Servizio: basic.hello.world
10 3) Immagine: busybox | Nome Servizio: busyboxsleep | Comando: sleep infinity
11 1
12 Manager> Schedulo  deploy  ==> Immagine: alpine | Nome Servizio: alpine.sleep |
   ↳ Comando: sleep infinity
13 Info> Task: alpine deployato (run) sul nodo: nodo2
14 nodo1 | UP | Task attivi(3):
15 nome : alpine.sleep-02ed572f73 | id : 578e6d5015ed | immagine : alpine:latest |
   ↳ comando : sleep infinity |
```

```

16 nome : alpine.sleep-3ea3bf8e09 | id : 1d9a09c5a8e9 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
17 nome : alpine.sleep-7a2ee9f608 | id : 5d5909aed749 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
18 nodo2 | UP | Task attivi(3):
19 nome : alpine.sleep-5e1ebb77c6 | id : e4e7aaa48bb2 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
20 nome : alpine.sleep-17e1ac63be | id : a06a19e76217 | immagine : alpine:latest |
   ↳ comando : sleep infinity |
21 nome : busyboxsleep-5ee4effa6f | id : 07ac0ad6fde8 | immagine : busybox:latest |
   ↳ comando : sleep infinity |
22 nodo3 | UP | Task attivi(0):

```

Riattivazione del nodo:

```

1 ---Menu Stato Nodi---
2 Scegli una voce dal menù:
3 1) Stampa stato e task attivi sui nodi
4 2) Stampa stato e task attivi e spenti sui nodi
5 3) Svuota un nodo
6 4) Attiva un nodo se è in DRAIN
7 5) Torna al Menu Principale
8 4
9 Scegli quale nodo vuoi attivare (sarà di nuovo considerato per il deploy)
10 1) nodo3
11 1
12 Manager> Schedulo attiva_nodo
13 Manager> Nodo nodo3 attivato, torna a poter ricevere nuovi task.

```

Adesso il nodo viene di nuovo considerato nell'assegnazione di nuove task:

```

1 ---Menu Servizi---
2 Scegli una voce dal menù:
3 1) Crea replica di 1 servizio
4 2) Scala N repliche di un servizio
5 3) Torna al Menu Principale
6 1
7 Scegli quale servizio avviare tra quelli disponibili:
8 1) Immagine: alpine | Nome Servizio: alpine.sleep | Comando: sleep infinity
9 2) Immagine: hello-world | Nome Servizio: basic.hello.world
10 3) Immagine: busybox | Nome Servizio: busyboxsleep | Comando: sleep infinity
11 1
12 Manager> Schedulo deploy ==> Immagine: alpine | Nome Servizio: alpine.sleep |
   ↳ Comando: sleep infinity
13 Info> Task: alpine deployato (run) sul nodo: nodo3
14 nodo1 | UP | Task attivi(3):
15 (...)
16 nodo2 | UP | Task attivi(3):
17 nome : alpine.sleep-5e1ebb77c6 | id : e4e7aaa48bb2 | immagine : alpine:latest |
   ↳ comando : sleep infinity (...)
18 nodo3 | UP | Task attivi(1):
19 nome : alpine.sleep-664aef4b49 | id : 8720442c9a35 | immagine : alpine:latest |
   ↳ comando : sleep infinity |

```

5.5 Caduta di un nodo

Per simulare la caduta di un nodo è possibile usare il comando "kill" sul container DinD, il thread `failure_detection` percepirà la caduta del nodo e reagirà:

```
1 docker kill nodo1
```

Questo era lo stato dei task prima della caduta del nodo:

```
1 nodo1 | UP | Task attivi(3):
2   (nome : alpine.sleep-02ed572f73 | id : 578e6d5015ed | immagine : alpine:latest |
   ↪ comando : sleep infinity |
3   nome : alpine.sleep-3ea3bf8e09 | id : 1d9a09c5a8e9 | immagine : alpine:latest |
   ↪ comando : sleep infinity |
4   nome : alpine.sleep-7a2ee9f608 | id : 5d5909aed749 | immagine : alpine:latest |
   ↪ comando : sleep infinity | )
5 nodo2 | UP | Task attivi(3):
6   (...)
7 nodo3 | UP | Task attivi(1):
8   (...)
```

Dopo la caduta del nodo si è attivato il meccanismo di failover, che ha mandato in coda i task del nodo1 per poterli eseguire sui nodi attivi:

```
1 Failure_detection> Il nodo nodo1 aveva questi task attivi e verranno rischedulati:
   ↪ [{'nome': 'alpine.sleep-02ed572f73', 'id': '578e6d5015ed', 'immagine':
   ↪ 'alpine:latest', 'comando': 'sleep infinity'}, {'nome':
   ↪ 'alpine.sleep-3ea3bf8e09', 'id': '1d9a09c5a8e9', 'immagine': 'alpine:latest',
   ↪ 'comando': 'sleep infinity'}, {'nome': 'alpine.sleep-7a2ee9f608', 'id':
   ↪ '5d5909aed749', 'immagine': 'alpine:latest', 'comando': 'sleep infinity'}]
2 Manager> Schedulo deploy ==> Immagine: alpine:latest | Nome Servizio:
   ↪ alpine.sleep | Comando: sleep infinity
3 nodo1 | DOWN
4 Info> Task: alpine:latest deployato (run) sul nodo: nodo3
5   (...)
6 nodo1 | DOWN
7 nodo2 | UP | Task attivi(4):
8   nome : alpine.sleep-bc5a25cc9f | id : 81d24b5c15c3 | immagine : alpine:latest |
   ↪ comando : sleep infinity |
9   nome : alpine.sleep-5e1ebb77c6 | id : e4e7aaa48bb2 | immagine : alpine:latest |
   ↪ comando : sleep infinity |
10  nome : alpine.sleep-17e1ac63be | id : a06a19e76217 | immagine : alpine:latest |
   ↪ comando : sleep infinity |
11  nome : busyboxsleep-5ee4effa6f | id : 07ac0ad6fde8 | immagine : busybox:latest |
   ↪ comando : sleep infinity |
12  nodo3 | UP | Task attivi(3):
13  nome : alpine.sleep-93a6d06d5b | id : 260e603e7a3a | immagine : alpine:latest |
   ↪ comando : sleep infinity |
14  nome : alpine.sleep-5021a0c20f | id : b9206207e4c0 | immagine : alpine:latest |
   ↪ comando : sleep infinity |
15  nome : alpine.sleep-664aef4b49 | id : 8720442c9a35 | immagine : alpine:latest |
   ↪ comando : sleep infinity |
16
```


6 Limiti e Sviluppi Futuri

Nonostante le funzionalità implementate, il sistema presenta alcuni limiti architetturali che potrebbero essere risolti con futuri miglioramenti.

Alcuni di questi limiti sono:

- Attualmente non è possibile eliminare voci dalla lista dei servizi disponibili o modificarli.
- La garbage collection si limita a ripulire i task spenti sul nodo, questo potrebbe essere esteso ad eliminare anche immagini, network e altri oggetti inutilizzati. Inoltre il fatto che i task spenti vengano eliminati in modo automatico non permette un'analisi approfondita sul fallimento del task, che aiuterebbe a non far ripresentare il problema ulteriormente.
- In questo momento i task consentono di specificare soltanto l'immagine e un comando da eseguire su di essa, è possibile espandere questa funzionalità includendo le limitazioni dell'uso della CPU, memoria disponibile per il task e le altre già disponibili grazie al Docker Engine.
- Ci sono dei limiti di sicurezza rappresentati dal fatto che i container DinD hanno bisogno di avere il flag `"--privileged"` attivo, il che fa perdere al docker host l'isolamento dai nodi stessi.

In futuro si potrebbe evolvere l'orchestratore per una gestione del sistema distribuito più simile a Docker Swarm con:

- Per lo scopo del progetto si è scelto di avere un ambiente contenuto con un numero statico di nodi, ma sarebbe possibile espandere questa funzionalità permettendo ad altri nodi di unirsi durante l'esecuzione previa richiesta al nodo manager.
- Aggiunta di un sistema di membership per gestire il sistema distribuito in modo più efficace, con la possibilità di avere più nodi manager che possano prendere il posto dell'orchestratore nel caso questo diventi inattivo.
- Si potrebbe aggiungere una funzionalità che permetta di includere anche i Dockerfile come servizi disponibili.
- Il nodo manager è in questo momento un punto di fallimento importante, se il nodo subisse un guasto il cluster perderebbe la capacità di schedare nuovi task o di monitorarsi. Per evitare questa situazione, Docker Swarm utilizza un algoritmo Raft Consensus tra i suoi nodi manager così da mantenere uno stato consistente dello stato dell'orchestratore. Se il nodo Leader Manager responsabile di schedare le task nel cluster cade, gli altri nodi manager decideranno un altro Leader Manager da eleggere all'interno del cluster per continuare l'esecuzione delle task in modo coordinato.[\[21\]](#)

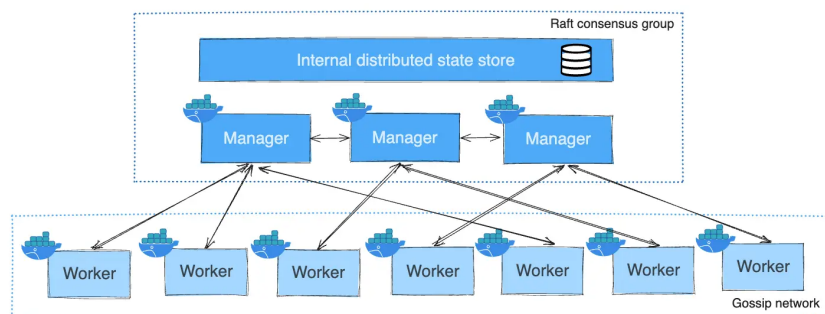


Figura 5: Nodi in Docker Swarm [\[17\]](#)

7 Conclusioni

Il progetto ha portato alla realizzazione di un prototipo di orchestratore, che riesce a gestire un sistema distribuito e l'obiettivo di gestire i cicli di vita dei container è stato raggiunto.

Si è riuscito a replicare diverse delle funzionalità dell'architettura di riferimento, per dimostrarlo ecco un confronto delle operazioni consentite dalla CLI di Docker Swarm rispetto all'orchestratore proposto:

- `swarm init` / `swarm join`, si è deciso di avere una infrastruttura statica tramite il deploy del cluster tramite `docker compose` l'orchestratore è subito pronto a gestire dei nodi worker su cui eseguire dei task.
- `service create`, è possibile creare un servizio in base alle necessità dell'utente che può essere eseguito come task sul cluster.
- `service scale` / `service rm`, è possibile decidere di scalare il numero di repliche di un determinato servizio con la possibilità di azzerare quelle repliche.
- `service ps`, l'orchestratore permette di mostrare i task che stanno attualmente venendo eseguiti sui vari nodi.
- `docker node ls` / `docker node ps`, tramite le funzionalità introdotte dall'orchestratore è possibile visualizzare la topologia del cluster, il loro stato attuale e i loro task attivi.
- `docker node update`, è permesso cambiare lo stato di un nodo per farlo passare in Drain così da svuotare i suoi task e passarli su altri nodi, impedendo al nodo di ricevere nuovi task. Così come è possibile renderlo nuovamente attivo per ricevere i task.

Il sistema soddisfa anche alcuni dei requisiti di un'architettura distribuita attraverso queste caratteristiche:

- Load Balancing, caratteristica garantita dalle politiche di scheduling least load del l'orchestratore che permette di distribuire i task tra i nodi in maniera equa in base al nodo con meno task attivi. Oltre allo scheduling most loaded per la rimozione di task sui nodi che hanno più task attivi al momento per bilanciare il sistema in modo efficace senza sovraccaricare i singoli nodi.
- Fault Tolerance, grazie all'uso di un heartbeat per controllare lo stato attuale di nodi è stato possibile riconoscere i fallimenti e usare una meccanica active-active failover per i nodi così da rispondere correttamente ai fallimenti tramite la esecuzione dei task su altri nodi.
- Garbage Collection, è stato possibile riconoscere task spente sui nodi così da prevenire l'accumulo di oggetti non utilizzati ed evitando il degrado delle prestazioni sui nodi worker.

Nonostante le semplificazioni adottate rispetto a Docker Swarm, il prototipo di orchestratore ha dimostrato di riuscire a gestire i cicli di vita dei container attivi sui nodi e riesce ad implementare alcune caratteristiche fondamentali dei sistemi distribuiti.

Riferimenti bibliografici

- [1] A. Silberschatz, P. B. Galvin, e G. Gagne, *Sistemi operativi. Concetti fondamentali*, 9a ed., Milano: Pearson, marzo 2014.
- [2] M. Van Steen e A. S. Tanenbaum, *Distributed Systems*, 4^a ed., Maarten van Steen, 2023.
- [3] S. Susnjara e I. Smalley, *What Is Container Orchestration?*, IBM Think, 22 ottobre 2024. [Online]. Disponibile: <https://www.ibm.com/think/topics/container-orchestration>.
- [4] Oracle Cloud Native, 8 dicembre 2025. [Online]. Disponibile: <https://www.oracle.com/se/cloud/cloud-native/container-registry/what-is-docker/>.
- [5] Docker Inc., *Docker Engine Security: Linux Namespaces*, Docker Documentation. [Online]. Disponibile: <https://docs.docker.com/engine/security/>.
- [6] Linux Man Pages, *namespaces(7) — overview of Linux namespaces*, Linux Programmer's Manual. [Online]. Disponibile: <https://man7.org/linux/man-pages/man7/namespaces.7.html>.
- [7] Linux Man Pages, *cgroups(7) — overview of Linux control groups*, Linux Programmer's Manual, 2026. [Online]. Disponibile: <https://man7.org/linux/man-pages/man7/cgroups.7.html>.
- [8] Docker Inc., *Docker-in-Docker and Containerized CI Workflows*, DockerCon 2023 Resources. [Online]. Disponibile: <https://www.docker.com/resources/docker-in-docker-containerized-ci-workflows-dockercon-2023/>.
- [9] J. Petazzoni, *Using Docker-in-Docker for your CI or testing environment? Think twice*, 2015. [Online]. Disponibile: <https://jpetazzo.github.io/2015/09/03/do-not-use-docker-in-docker-for-ci/>.
- [10] Docker Inc., *Swarm mode overview*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/engine/swarm/>.
- [11] Docker Inc., *Swarm mode key concepts*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/engine/swarm/key-concepts/>.
- [12] J. J. Smile, *Docker Swarm Architecture and Concepts*, DevOps Notes, 2021. [Online]. Disponibile: <https://smilejj91.github.io/devops/Docker-swarm/>.
- [13] ActiveLobby Team, *What is Docker Swarm*, ActiveLobby Blogs, 2020. [Online]. Disponibile: <https://blog.supportlobby.com/what-is-docker-swarm/>.
- [14] Docker Inc., *Docker Swarm CLI Reference*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/reference/cli/docker/swarm/>.
- [15] Docker Inc., *Docker Service CLI Reference*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/reference/cli/docker/service/>.
- [16] Docker Inc., *Docker Node CLI Reference*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/reference/cli/docker/node/>.
- [17] Docker Inc., *Manage nodes in a swarm*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/engine/swarm/manage-nodes/>.

- [18] MathWorks, *What Is Fault Detection, Isolation, and Recovery (FDIR)?*, MathWorks Discovery Pages, 2026. [Online]. Disponibile: <https://in.mathworks.com/discovery/fdir.html>.
- [19] Docker Inc., *How Swarm services work*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/engine/swarm/how-swarm-mode-works/services/>.
- [20] Docker Inc., *Prune unused Docker objects*, Docker Documentation, 2026. [Online]. Disponibile: <https://docs.docker.com/engine/manage-resources/pruning/>.
- [21] Docker Documentation, *Raft consensus algorithm in Swarm mode*. Disponibile online: <https://docs.docker.com/engine/swarm/raft/>